



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Τομέας Τεχνολογίας Πληροφορικής & Υπολογιστών

7^ο Εξάμηνο: Εργαστήριο Λειτουργικών Συστημάτων

3^η Άσκηση: Συστήματα Αρχείων σε Περιβάλλον Linux

Ημερομηνία Παρουσίασης: 9/1/2025

Γιάννης Πολυχρονόπουλος

Αριθμός Μητρώου: 03121089

Κωνσταντίνος Σταυλιώτης

Αριθμός Μητρώου: 03121104

1^ο Μέρος

Η εικόνα fsdisk1.img

Ερώτηση 1

Παρατηρούμε πως για να τρέξουμε την εικονική μηχανή μας στο τέλος του utopia.sh script εκτελείται (exec) το εκτελέσιμο qemu-system-x86_64. Επίσης παρατηρούμε πως με την εντολή αυτή έχουν παρθεί πολλές παραμέτρους (σημαίες). Για να προσθέσουμε στην εικονική μηχανή utopia έναν επιπλέον δίσκο για την εικόνα fsdisk1.img θα προσθέσουμε ακόμα μια σημαία στην εντολή, την σημαία **drive**.

```
QEMU_BUILD_DIR=$HOME/opt/qemu
QEMU=$QEMU_BUILD_DIR/bin/qemu-system-x86_64
```

```
exec $QEMU -enable-kvm -M pc -m $UTOPIA_MEMORY_MB \
-drive file=./ext2-vdisks/fsdisk1-7c2b425a6.img,format=raw,if=virtio \
-drive file=./ext2-vdisks/fsdisk2-a0173283d.img,format=raw,if=virtio \
-drive file=./ext2-vdisks/fsdisk3-982902777.img,format=raw,if=virtio \
```

Η επιλογή -drive στη γραμμή εντολών του QEMU χρησιμοποιείται για να καθορίσουμε ένα μέσο αποθήκευσης (π.χ. δίσκος, ISO, εικόνα δίσκου) που θα προσαρτηθεί ως συσκευή στο εικονικό μηχάνημα. Για την επιλογή αυτή χρησιμοποιήθηκαν οι παρακάτω παραμέτροι.

file=./ext2-vdisks/fsdisk1-7c2b425a6.img

Ορίζουμε το αρχείο που θέλουμε ως επιπλέον δίσκο στην εικονική μας μηχανή.

format=raw

Δηλώνουμε τη μορφή του αρχείου εικόνας δίσκου. Η μορφή raw υποδηλώνει ότι το αρχείο περιέχει έναν ωμό (raw) δίσκο, δηλαδή τα δεδομένα του είναι αποθηκευμένα ακριβώς όπως βρίσκονται στον φυσικό δίσκο.

if=virtio

Καθορίζουμε τον τύπο του ελεγκτή (controller interface) που χρησιμοποιείται για να συνδέσουμε το μέσο στο εικονικό μηχάνημα. Το virtio είναι ένας παραμετροποιημένος τύπος ελεγκτή υψηλής απόδοσης που σχεδιάστηκε για εικονικές μηχανές, μειώνοντας την υπερφόρτωση του virtualization (overhead).

Σημειώνεται ότι όλες οι επιλογές -drive πρέπει να δηλωθούν μετά από τις -enable-kvm, -M, -m και -smp σημαίες, διαφορετικά δε θα λειτουργήσει σωστά η εικονική μηχανή.

Επομένως, θα έχουμε τώρα τις ακόλουθες συσκευές:

```
root@utopia: # lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
fd0 2:0 1 4K 0 disk
sr0 11:0 1 1024M 0 rom
vda 254:0 0 10G 0 disk
└─vda1 254:1 0 10G 0 part /
vdb 254:16 0 50M 0 disk
vdc 254:32 0 20M 0 disk
vdd 254:48 0 20M 0 disk
```

Άρα έχουμε τις συσκευές vdb (fsdisk1), vdc (fsdisk2), vdd (fsdisk3).

Ερώτηση 2

Δημιουργούμε αρχείο που θα προσαρτήσουμε το img και κάνουμε την προσάρτηση.

```
root@utopia: # mkdir /mnt/fsdisk1 && mount /dev/vdb /mnt/fsdisk1
```

Έπειτα, χρησιμοποιούμε την εντολή df για να καθορίσουμε το μέγεθος του δίσκου που προσαρτήσαμε στο

σύστημά μας. Μπορούμε να δούμε ότι έχει μέγεθος 50 MB.

```
root@utopia: # df -h /mnt/fsdisk1/
Filesystem Size Used Avail Use% Mounted on
/dev/vdb 49M 16K 46M 1% /mnt/fsdisk1
```

Τι είναι η εντολή 'df'?

Η εντολή 'df' σημαίνει «disk free» παρέχει ένα στιγμιότυπο της χρήσης του δίσκου, δείχνοντας τον συνολικό χώρο, τον χρησιμοποιημένο χώρο, τον διαθέσιμο χώρο και το ποσοστό χρήσης για κάθε προσαρτημένο σύστημα αρχείων. Η επιλογή -h εκτυπώνει μεγέθη σε human-readable format (π.χ. KB, MB, GB).

Hexedit

Καλούμε την εντολή hexedit /dev/vdb. Εμφανίζεται ένας editor, στο κάτω μέρος του οποίου βλέπουμε ότι υπάρχουν συνολικά 0x3200000 bytes στην εικόνα /dev/vdb. Αυτό αντιστοιχεί σε 50 MB, άρα αυτό είναι και το μέγεθος του δίσκου μας.

Ερώτηση 3

Για να βρούμε το σύστημα αρχείων που περιέχει αρκεί να χρησιμοποιήσουμε την εντολή blkid. Η εντολή αυτή χρησιμοποιείται για την εμφάνιση πληροφοριών συσκευής μπλοκ σε ένα σύστημα Linux. Η εντολή blkid λειτουργεί διαβάζοντας και ερμηνεύοντας τα μεταδεδομένα που είναι αποθηκευμένα στις επικεφαλίδες των συσκευών μπλοκ. Κάθε υποστηριζόμενο σύστημα αρχείων έχει έναν μοναδικό «magic number», δηλαδή μια ακολουθία bytes που αποθηκεύεται σε σταθερή θέση στα μεταδεδομένα και προσδιορίζει τον τύπο του περιεχομένου. Το blkid χρησιμοποιεί μια βάση δεδομένων με γνωστούς μαγικούς αριθμούς και τα αντίστοιχα συστήματα αρχείων για να προσδιορίσει τον τύπο.

```
root@utopia: # blkid /dev/vdb
/dev/vdb: LABEL="fsdisk1.img" UUID="c63028e5-711b-410d-a263-e7ca2b15a8d3" BLOCK_SIZE="1024"
TYPE="ext2"
```

Hexedit

Για να βρούμε το σύστημα αρχείων που περιέχει ο δίσκος πρέπει να μεταβούμε στο superblock, στο offset 0x38 και να διαβάσουμε το magic number. Για τα συστήματα αρχείων ext2, ext3 και ext4, το magic number είναι ο αριθμός 0xEF53. Για να βρούμε όμως αυτή τη θέση πρέπει να γνωρίζουμε το block size του δίσκου καθώς το superblock είναι το δεύτερο block μετά το boot sector. Υποθέτοντας ότι το σύστημα αρχείων είναι ένα από τα ext2/ext3/ext4, θα αναζητήσουμε το magic number στις θέσεις 0x438, 0x838 και 0x1038, καθώς αν το σύστημα αρχείων έχει block size 1024, 2048 ή 4096 bytes, η πρώτη διεύθυνση του superblock θα είναι στη θέση 0x400, 0x800 ή 0x1000 αντίστοιχα. Παρατηρούμε ότι οι θέσεις 0x838 και 0x1038 είναι 0, ενώ η 0x438 περιέχει τον αριθμό 0xEF53 επιβεβαιώνοντας ότι πρόκειται για το σύστημα αρχείων ext2/ext3/ext4 και ότι το block size είναι 1024 bytes, συνεπώς το superblock ξεκινάει από το offset 0x400.

Ερώτηση 4

Για τις επόμενες ερωτήσεις θα χρησιμοποιήσουμε την εντολή **tune2fs**, που επιτρέπει να δούμε και να αλλάξουμε τις παραμέτρους ενός συστήματος αρχείου. Η επιλογή -l χρησιμοποιείται με την εντολή 'tune2fs' στο Linux για την εμφάνιση πληροφοριών του συστήματος αρχείων. Επομένως, για τα επόμενα ερωτήματα απλώς ψάχνουμε σε αυτή την λίστα τις πληροφορίες που αναζητούμε:

```
root@utopia: /# tune2fs -l /dev/vdb | grep "Filesystem created:"
Filesystem created: Tue Dec 12 15:23:16 2023
```

Hexedit

Το offset 0x440 περιγράφει την τελευταία φορά που έγινε έλεγχος στο δίσκο με τη χρήση εργαλείων όπως το fsck. Εφόσον δεν έχουμε κάνει κάποιον τέτοιο έλεγχο, αυτό το timestamp απευθύνεται στη χρονική στιγμή δημιουργίας του δίσκου. Διαβάζουμε τον αριθμό 0x65787AE4 ή 1702394596. Για την εξαγωγή της ημερομηνίας

χρησιμοποιούμε την εντολή:

```
TZ=UTC date -d @1702394596
```

Προκύπτει ότι το σύστημα αρχείων δημιουργήθηκε την Τρίτη, 12/12/2023, 15:23:17 UTC.

Ερώτηση 5

```
root@utopia:/# tune2fs -l /dev/vdb | grep "Last mount time:"  
Last mount time: Sun Dec 29 16:02:19 2024
```

Hexedit

Το last mount time βρίσκεται στο offset 0x42C. Διαβάζουμε 0x6777D02F. Με την ίδια εντολή που χρησιμοποιήσαμε και παραπάνω, βλέπουμε ότι η τελευταία προσάρτηση έγινε την Παρασκευή, 3/1/2025, 11:55:27 UTC.

Ερώτηση 6

```
root@utopia:/# tune2fs -l /dev/vdb | grep "Last mounted on:"  
Last mounted on: /mnt/fsdisk1
```

Hexedit

Το μονοπάτι στο οποίο προσαρτήθηκε τελευταία φορά ο δίσκος μπορεί να βρεθεί στο offset 0x488 - 0x493. Μπορούμε να δούμε από την ερμηνεία σε ASCII ότι είναι το σημείο /mnt/fsdisk1.

Ερώτηση 7

```
root@utopia:/# tune2fs -l /dev/vdb | grep "Last write time:"  
Last write time: Sun Dec 29 16:02:19 2024
```

Hexedit

Το offset 0x430 αντιστοιχεί στο last write time. Παρατηρούμε ότι είναι το ίδιο timestamp με το last mount time, καθώς δεν έχουμε κάνει καμία εγγραφή στο δίσκο.

Ερώτηση 8

Ένα block σε ένα σύστημα αρχείων είναι η μικρότερη μονάδα αποθήκευσης δεδομένων, με τυπικά μεγέθη όπως 512 bytes, 1 KB, 4 KB, ή 8 KB. Το μέγεθός του επηρεάζει την αποδοτικότητα και την απόδοση: μικρότερα μπλοκ μειώνουν τη σπατάλη χώρου για μικρά αρχεία αλλά αυξάνουν τη διαχείριση, ενώ μεγαλύτερα μπλοκ βελτιώνουν την ταχύτητα ανάγνωσης/εγγραφής για μεγάλα αρχεία αλλά αυξάνουν τον κατακερματισμό και τη σπατάλη χώρου εάν υπάρχουν πολλά μικρά αρχεία.

Ερώτηση 9

Συνεχίζουμε να αντλούμε πληροφορίες για το σύστημα αρχείου από την εντολή tune2fs. **root@utopia:/# tune2fs -l /dev/vdb | grep "Block size:"**
Block size: 1024

Hexedit

Η πληροφορία βρίσκεται στο offset 0x418. Παρατηρούμε ότι η θέση περιέχει τον αριθμό 0. Μπορούμε να υπολογίσουμε το block size ως εξής:

$$Size = 2^{10+0} = 1024$$

Ερώτηση 10

Ένα **inode** είναι μια δομή δεδομένων σε ένα σύστημα αρχείων που αναπαριστά κάθε "αντικείμενο" μέσα στο σύστημα, όπως κανονικά αρχεία, καταλόγους, συμβολικούς συνδέσμους και ειδικά αρχεία (π.χ. FIFO, block, χαρακτήρες, socket). Το inode περιέχει όλα τα μεταδεδομένα του αντικειμένου, εκτός από το όνομα και τα πραγματικά δεδομένα.

Ερώτηση 11

Αξιοποιούμε πάλι την εντολή **tune2fs**:

```
root@utopia:/# tune2fs -l /dev/vdb | grep "Inode size:"  
Inode size: 128
```

Hexedit

Στο offset 0x458 διαβάζουμε 0x0080 = 128 bytes.

Ερώτηση 12

```
root@utopia:/# tune2fs -l /dev/vdb | grep -E "Free blocks|Free inodes"  
Free blocks: 49552  
Free inodes: 12810
```

Hexedit

Τα διαθέσιμα blocks βρίσκονται στη θέση 0x40C και τα διαθέσιμα inodes βρίσκονται στη θέση 0x410. Προκύπτει ότι υπάρχουν 49552 διαθέσιμα blocks και 12810 διαθέσιμα inode στο δίσκο.

Ερώτηση 13

Το **superblock** είναι μια κρίσιμη δομή δεδομένων σε ένα σύστημα αρχείων που περιέχει πληροφορίες για το ίδιο το σύστημα αρχείων. Βρίσκεται αποθηκευμένο σε συγκεκριμένη περιοχή του δίσκου και παίζει κεντρικό ρόλο στη διαχείριση και λειτουργία του συστήματος αρχείων. Περιέχει δεδομένα όπως:

1. Μέγεθος του συστήματος αρχείων.
2. Μέγεθος του block που χρησιμοποιείται.
3. Θέση κρίσιμων δομών, όπως τα bitmaps και οι πίνακες inode.
4. Στατιστικά και τύπος συστήματος αρχείων.

Ερώτηση 14

Το superblock ορίζεται σε κάθε block group στο πρώτο block. Περιέχει πληροφορίες όπως ο συνολικός αριθμός των μπλοκ στο δίσκο, το μέγεθος ενός μπλοκ (1024 bytes), ο αριθμός των ελεύθερων μπλοκ κ.λπ. Ξεκινά σε μια μετατόπιση 1024 bytes από την αρχή του δίσκου, δηλαδή μετά το boot sector, και εκτείνεται σε 1 block.

Ερώτηση 15

Η ύπαρξη εφεδρικών αντιγράφων του **superblock** στο σύστημα αρχείων **ext2** έχει νόημα για λόγους **ακεραιότητας, αξιοπιστίας και ανάκαμψης από σφάλματα**. Εάν το αρχικό superblock καταστραφεί ή αλλοιωθεί (π.χ., λόγω προβλημάτων στο υλικό, απρόσμενων διακοπών ρεύματος ή λογισμικών σφαλμάτων), το σύστημα αρχείων γίνεται μη προσβάσιμο. Τα εφεδρικά αντίγραφα του superblock τοποθετούνται σε προκαθορισμένα σημεία του δίσκου και επιτρέπουν την **επανόρθωση** της δομής του συστήματος αρχείων μέσω εργαλείων όπως το **fsck** (filesystem check).

Ερώτηση 16

Η εντολή `dumpe2fs` χρησιμοποιείται για την εκτύπωση των πληροφοριών super block και blocks group για το σύστημα αρχείων που υπάρχει στη συσκευή. Εμείς θέλουμε τα αποθηκευμένα εφεδρικά αντίγραφα του superblock, επομένως:

```
root@utopia:/# dumpe2fs /dev/vdb | grep -i "Backup superblock"
dumpe2fs 1.47.0 (5-Feb-2023)
Backup superblock at 8193, Group descriptors at 8194-8194
Backup superblock at 16385, Group descriptors at 16386-16386
Backup superblock at 24577, Group descriptors at 24578-24578
Backup superblock at 32769, Group descriptors at 32770-32770
Backup superblock at 40961, Group descriptors at 40962-40962
Backup superblock at 49153, Group descriptors at 49154-49154
```

Hexedit

Μεταβαίνουμε στη διεύθυνση 0x800, η οποία είναι η πρώτη θέση του block group descriptor. Κάθε 32 bytes περιγράφουν ένα block group, συνεπώς μπορούμε να βρούμε όλες τις block διευθύνσεις στις οποίες υπάρχει ένα block bit map. Διαβάζουμε 0x0003, 0x2003, 0x4003, 0x6003, 0x8003, 0xA003, 0xC003. Συμπεραίνουμε ότι το superblock είναι το πρώτο block bit map στην διεύθυνση 3 και εφόσον το superblock βρίσκεται στην block διεύθυνση 1, τα block group descriptors έχουν όλα μέγεθος 1 block. Άρα, όλα τα superblocks βρίσκονται δύο blocks πίσω από τα block bit maps. Συνεπώς, τα εφεδρικά superblocks βρίσκονται στις block διευθύνσεις 0x2001 = 8193, 0x4001 = 16385, 0x6001 = 24577, 0x8001 = 32769, 0xA001 = 40961, 0xC001 = 49153.

Ερώτηση 17

Το Block group αποτελεί μία σειρά από συνεχόμενα blocks. Είναι μια βασική υποδιαίρεση του δίσκου, που οργανώνει τα δεδομένα σε μικρότερες, πιο διαχειρίσιμες ενότητες. Κάθε block group λειτουργεί ως ανεξάρτητη μονάδα που περιέχει όλα τα απαραίτητα στοιχεία για τη διαχείριση των δεδομένων του. Το block περιέχει:

1. Superblock - 1 μπλοκ
2. Block group descriptors - N μπλοκ (ανάλογα με τον αριθμό των block groups)
3. Block bitmap - 1 μπλοκ
4. Inode bitmap - 1 μπλοκ
5. Inode table - N μπλοκ (ανάλογα με το μέγεθος του inode και το μέγεθος του μπλοκ)
6. Data blocks - N μπλοκ (ανάλογα με το μέγεθος του μπλοκ και το μέγεθος των block groups)

Ερώτηση 18

Αρχικά στο partition που θα βρίσκεται το σύστημα αρχείου μας τα πρώτα 1024 bytes κατανέμονται στο boot sector. Τα υπόλοιπα αποτελούνται από block groups. Το πλήθος των block groups σε ένα σύστημα αρχείων ext2 εξαρτάται από το μέγεθος του συστήματος αρχείων και το μέγεθος κάθε block group. Ο αριθμός των block groups υπολογίζεται με τον τύπο:

$$\text{Αριθμός Block Groups} = \frac{\text{Συνολικά Blocks στο Filesystem}}{\text{Blocks per Group}}$$

1. **Συνολικά Blocks στο Filesystem:** Προκύπτει από το συνολικό μέγεθος του δίσκου (filesystem size) διαιρεμένο με το μέγεθος του block.
2. **Blocks per Group:** Ορίζεται κατά τη δημιουργία του συστήματος αρχείων και εξαρτάται από το μέγεθος του block (1 KB, 2 KB, 4 KB κ.λπ.).

Τα block groups είναι οργανωμένα συνεχόμενα και διαχειρίζονται δεδομένα ανεξάρτητα, προσφέροντας καλύτερη απόδοση και ταχύτερη πρόσβαση στα δεδομένα.

Ερώτηση 19

Αξιοποιούμε την εντολή **dumpe2fs**

```
root@utopia:/mnt/fsdisk1# dumpe2fs /dev/vdb
Group 0: (Blocks 1-8192)
Primary superblock at 1, Group descriptors at 2-2
Block bitmap at 3 (+2)
Inode bitmap at 4 (+3)
Inode table at 5-233 (+4)
7944 free blocks, 1821 free inodes, 2 directories
Free blocks: 248-1024, 1026-8192
Free inodes: 12-1832
Group 1: (Blocks 8193-16384)
Backup superblock at 8193, Group descriptors at 8194-8194
Block bitmap at 8195 (+2)
Inode bitmap at 8196 (+3)
Inode table at 8197-8425 (+4)
7958 free blocks, 1831 free inodes, 1 directories
Free blocks: 8427-16384
Free inodes: 1834-3664
Group 2: (Blocks 16385-24576)
Backup superblock at 16385, Group descriptors at 16386-16386
Block bitmap at 16387 (+2)
Inode bitmap at 16388 (+3)
Inode table at 16389-16617 (+4)
7959 free blocks, 1832 free inodes, 0 directories
Free blocks: 16618-24576
Free inodes: 3665-5496
Group 3: (Blocks 24577-32768)
Backup superblock at 24577, Group descriptors at 24578-24578
Block bitmap at 24579 (+2)
Inode bitmap at 24580 (+3)
Inode table at 24581-24809 (+4)
7959 free blocks, 1832 free inodes, 0 directories
Free blocks: 24810-32768
Free inodes: 5497-7328
Group 4: (Blocks 32769-40960)
Backup superblock at 32769, Group descriptors at 32770-32770
Block bitmap at 32771 (+2)
Inode bitmap at 32772 (+3)
Inode table at 32773-33001 (+4)
7959 free blocks, 1832 free inodes, 0 directories
Free blocks: 33002-40960
Free inodes: 7329-9160
Group 5: (Blocks 40961-49152)
Backup superblock at 40961, Group descriptors at 40962-40962
Block bitmap at 40963 (+2)
Inode bitmap at 40964 (+3)
Inode table at 40965-41193 (+4)
7959 free blocks, 1830 free inodes, 1 directories
Free blocks: 41194-49152
Free inodes: 9163-10992
Group 6: (Blocks 49153-51199)
Backup superblock at 49153, Group descriptors at 49154-49154
Block bitmap at 49155 (+2)
Inode bitmap at 49156 (+3)
Inode table at 49157-49385 (+4)
1814 free blocks, 1832 free inodes, 0 directories
Free blocks: 49386-51199
```

Free inodes: 10993-12824

Από την έξοδο της εντολής **dumpe2fs**, μπορούμε να δούμε ότι το σύστημα αρχείων περιέχει **7 block groups**.

Group 0: (Blocks 1–8192)

Group 1: (Blocks 8193–16384)

Group 2: (Blocks 16385–24576)

Group 3: (Blocks 24577–32768)

Group 4: (Blocks 32769–40960)

Group 5: (Blocks 40961–49152)

Group 6: (Blocks 49153–51199)

Hexedit

Από το ερώτημα 16 μπορούμε να μετρήσουμε 6 εφεδρικά superblocks άρα 7 block group descriptors συνολικά.

Ερώτηση 20

Ο **Block Group Descriptor (BGD)** στο σύστημα αρχείων **ext2** είναι μια δομή δεδομένων που παρέχει σημαντικές πληροφορίες για κάθε ομάδα μπλοκ (*block group*). Ο ρόλος του Block Group Descriptor είναι να αποθηκεύει μεταδεδομένα που περιγράφουν τις κρίσιμες δομές και την οργάνωση δεδομένων μιας συγκεκριμένης ομάδας μπλοκ.

Κάθε block group descriptor αποθηκεύει τα παρακάτω μεταδεδομένα για το block group:

1. Θέση του block bitmap του block group
2. Θέση του inode bitmap του block group
3. Θέση του inode table μέσα στο block group
4. Αριθμός διαθέσιμων blocks στο block group
5. Αριθμός διαθέσιμων inodes στο block group
6. Αριθμός καταλόγων στο block group

Ερώτηση 21

Η ύπαρξη εφεδρικών αντιγράφων των **block group descriptors (BGDs)** στο σύστημα αρχείων **ext2** έχει το ίδιο νόημα που έχει και η ύπαρξη εφεδρικών αντιγράφων του **superblock**. Δηλαδή, προσδίδει ανθεκτικότητα και αξιοπιστία σε περίπτωση βλάβης ή καταστροφής δεδομένων σε κάποιο μέρος του δίσκου.

Ερώτηση 22

Σύμφωνα με την έξοδο της εντολής **dumpe2fs** τα εφεδρικά αντίγραφα των **block group descriptors** αποθηκεύονται στα παρακάτω μπλοκ:

Μπλοκ με εφεδρικά αντίγραφα των block group descriptors:

1. **Group 1:**
 - **Backup superblock:** Block 8193.
 - **Block group descriptors:** Block 8194.
2. **Group 2:**
 - **Backup superblock:** Block 16385.
 - **Block group descriptors:** Block 16386.

3. Group 3:

- **Backup superblock:** Block **24577**.
- **Block group descriptors:** Block **24578**.

4. Group 4:

- **Backup superblock:** Block **32769**.
- **Block group descriptors:** Block **32770**.

5. Group 5:

- **Backup superblock:** Block **40961**.
- **Block group descriptors:** Block **40962**.

6. Group 6:

- **Backup superblock:** Block **49153**.
- **Block group descriptors:** Block **49154**.

Παρατήρηση:

Κάθε block group πέρα από το πρώτο (**Group 0**) περιέχει αντίγραφα του **superblock** και των **block group descriptors** για πλεονασμό και ανθεκτικότητα σε περίπτωση βλάβης.

Hexedit

Από την ανάλυση στο ερώτημα 16 προκύπτει ότι τα εφεδρικά αντίγραφα των block group descriptors βρίσκονται στα blocks 8194, 16386, 24578, 32770, 40962 και 49154.

Ερώτηση 23

Το **block bitmap** και το **inode bitmap** είναι δύο κρίσιμες δομές στο σύστημα αρχείων **ext2**, που χρησιμοποιούνται για τη διαχείριση της κατανομής μπλοκ και inodes αντίστοιχα.

- **Block Bitmap:** Είναι ένας χάρτης bits όπου κάθε bit αντιπροσωπεύει ένα μπλοκ δεδομένων στην ομάδα μπλοκ (block group). Αν το bit είναι **1**, το μπλοκ είναι δεσμευμένο, ενώ αν είναι **0**, το μπλοκ είναι ελεύθερο. Ένα block bitmap μπορεί να διαχειριστεί έως και 8 φορές τον αριθμό των bits που περιέχει το μέγεθος ενός μπλοκ (π.χ., για μπλοκ 1 KB, $1024 \times 8 = 8192$ μπλοκ ανά block group). Τα bits αποθηκεύονται σε little-endian μορφή, όπου το LSB (Least Significant Bit) αντιπροσωπεύει το μικρότερο block index.
- **Inode Bitmap:** Λειτουργεί με τον ίδιο τρόπο, αλλά κάθε bit αντιπροσωπεύει ένα inode. Αν το bit είναι **1**, το inode είναι δεσμευμένο (π.χ., χρησιμοποιείται από ένα αρχείο ή φάκελο), ενώ αν είναι **0**, είναι διαθέσιμο για χρήση.

Πού βρίσκονται:

Και τα δύο bitmaps αποθηκεύονται μέσα σε κάθε ομάδα μπλοκ (block group), αμέσως μετά το **block group descriptor table**. Ο ακριβής τους εντοπισμός προσδιορίζεται από το superblock και τα metadata του filesystem.

Αυτή η διάταξη εξασφαλίζει ότι κάθε ομάδα μπλοκ είναι αυτοδύναμη στη διαχείριση των μπλοκ δεδομένων και των inodes της, ενισχύοντας την αποδοτικότητα και την ανθεκτικότητα του συστήματος αρχείων.

```
root@utopia:/mnt/fsdisk1# dumpe2fs /dev/vdb | grep -i "Block bitmap" -A1
dumpe2fs 1.47.0 (5-Feb-2023)
Block bitmap at 3 (+2)
Inode bitmap at 4 (+3)
-
Block bitmap at 8195 (+2)
Inode bitmap at 8196 (+3)
```

```

-
Block bitmap at 16387 (+2)
Inode bitmap at 16388 (+3)
-
Block bitmap at 24579 (+2)
Inode bitmap at 24580 (+3)
-
Block bitmap at 32771 (+2)
Inode bitmap at 32772 (+3)
-
Block bitmap at 40963 (+2)
Inode bitmap at 40964 (+3)
-
Block bitmap at 49155 (+2)
Inode bitmap at 49156 (+3)

```

Hexedit

Ομοίως με το προηγούμενο ερώτημα, η ανάγνωση του πρώτου block group descriptor επιστρέφει τα εξής αποτελέσματα:

Block bit map blocks: 3, 8195, 16387, 24579, 32771, 40963, 49155 Inode bit map blocks: 4, 8196, 16388, 24580, 32772, 40964, 49156

Ερώτηση 24

Τα **inode tables** είναι πίνακες που περιέχουν τις δομές δεδομένων των inodes, οι οποίες αποθηκεύουν πληροφορίες για τα αρχεία και τους φακέλους στο σύστημα αρχείων ext2. Κάθε inode περιέχει δεδομένα όπως τα δικαιώματα πρόσβασης, το μέγεθος του αρχείου, τις χρονικές σφραγίδες και δείκτες στα μπλοκ δεδομένων.

Χαρακτηριστικά:

- Κάθε inode έχει μέγεθος (παραδοσιακά 128 bytes, αλλά μπορεί να είναι μεγαλύτερο σε νέες εκδόσεις).
- Η αρίθμηση των inodes ξεκινά από το **1**.
- Ορισμένα inodes είναι δεσμευμένα από το σύστημα αρχείων (π.χ., το **inode #2** για τον root κατάλογο και το **inode #11** για τον φάκελο **lost+found**).

Τοποθεσία στον δίσκο:

Τα inode tables βρίσκονται σε κάθε ομάδα μπλοκ (block group), μετά τα **block bitmap** και **inode bitmap**, καταλαμβάνοντας έναν λογικά συνεχόμενο χώρο από μπλοκ. Η τοποθεσία τους καθορίζεται από το **block group descriptor** και η διαρρύθμισή τους διασφαλίζει ότι κάθε ομάδα μπλοκ διαχειρίζεται ανεξάρτητα τα inodes της.

```

root@utopia:/mnt/fsdisk1# dumpe2fs /dev/vdb | grep -i "Inode table"
dumpe2fs 1.47.0 (5-Feb-2023)
Inode table at 5-233 (+4)
Inode table at 8197-8425 (+4)
Inode table at 16389-16617 (+4)
Inode table at 24581-24809 (+4)
Inode table at 32773-33001 (+4)
Inode table at 40965-41193 (+4)
Inode table at 49157-49385 (+4)

```

Hexedit

Μεταβαίνουμε στο offset 0x428, στο superblock, όπου βρίσκουμε ότι τα inodes per group είναι 1832. Υπο-

λογίζουμε πόσα blocks υπάρχουν στο inode table

$$\text{Number of blocks} = \frac{\text{inodes per group} \cdot \text{inode size}}{\text{block}} = \frac{1832 \cdot 128}{1024} = 229$$

Από το block group descriptor μπορούμε να εξάγουμε την πρώτη block διεύθυνση κάθε inode table, συνεπώς προσθέτοντας 228 μπορούμε να υπολογίσουμε όλες τις block διευθύνσεις που αποτελούν όλα τα inode tables:

5 - 233, 8197 - 8425, 16389 - 16617, 24581 - 24809, 32773 - 33001, 40965 - 41193, 49517 - 49385.

Ερώτηση 25

Inode Data Structure

Starting Byte	Ending Byte	Size in Bytes	Field Description
0	1	2	Type and Permissions (see below)
2	3	2	User ID
4	7	4	Lower 32 bits of size in bytes
8	11	4	Last Access Time (in POSIX time)
12	15	4	Creation Time (in POSIX time)
16	19	4	Last Modification time (in POSIX time)
20	23	4	Deletion time (in POSIX time)
24	25	2	Group ID
26	27	2	Count of hard links (directory entries) to this inode. When this reaches 0, the data blocks are marked as unallocated.
28	31	4	Count of disk sectors (not Ext2 blocks) in use by this inode, not counting the actual inode structure nor directory entries linking to the inode.
32	35	4	Flags (see below)
36	39	4	Operating System Specific value #1
40	43	4	Direct Block Pointer 0
44	47	4	Direct Block Pointer 1
48	51	4	Direct Block Pointer 2
52	55	4	Direct Block Pointer 3
56	59	4	Direct Block Pointer 4
60	63	4	Direct Block Pointer 5
64	67	4	Direct Block Pointer 6
68	71	4	Direct Block Pointer 7
72	75	4	Direct Block Pointer 8
76	79	4	Direct Block Pointer 9
80	83	4	Direct Block Pointer 10
84	87	4	Direct Block Pointer 11
88	91	4	Singly Indirect Block Pointer (Points to a block that is a list of block pointers to data)
92	95	4	Doubly Indirect Block Pointer (Points to a block that is a list of block pointers to Singly Indirect Blocks)
96	99	4	Triply Indirect Block Pointer (Points to a block that is a list of block pointers to Doubly Indirect Blocks)
100	103	4	Generation number (Primarily used for NFS)
104	107	4	In Ext2 version 0, this field is reserved. In version >= 1, Extended attribute block (File ACL).
108	111	4	In Ext2 version 0, this field is reserved. In version >= 1, Upper 32 bits of file size (if feature bit set) if it's a file, Directory ACL if it's a directory
112	115	4	Block address of fragment
116	127	12	Operating System Specific Value #2

Πού αποθηκεύεται μέσα στον δίσκο:

Τα **inodes** αποθηκεύονται στον **πίνακα inodes (inode table)**, ο οποίος βρίσκεται σε κάθε ομάδα μπλοκ

(block group) του συστήματος αρχείων. Συγκεκριμένα, ο **πίνακας inodes** είναι ένας λογικός συνεχής χώρος από μπλοκ που έπεται των **block bitmap** και **inode bitmap**. Η θέση του πίνακα inodes καθορίζεται από τον **block group descriptor**.

Ερώτηση 26

Μπλοκ ανά Block Group:

```
root@utopia:/mnt/fsdisk1# dumpe2fs /dev/vdb | grep -i "Blocks per group"
dumpe2fs 1.47.0 (5-Feb-2023)
Blocks per group: 8192
```

- Κάθε block group καλύπτει μια διαδοχική περιοχή **8192 μπλοκ** (π.χ., από Block 1–8192 για το Group 0, από Block 8193–16384 για το Group 1 κ.λπ.).
- Άρα, κάθε block group περιέχει 8192 μπλοκ.

Inodes ανά Block Group:

```
root@utopia:/mnt/fsdisk1# dumpe2fs /dev/vdb | grep -i "Inodes per group"
dumpe2fs 1.47.0 (5-Feb-2023)
Inodes per group: 1832
```

Hexedit

Γνωρίζουμε ήδη ότι inodes per group = 1832. Στο offset 32 (0x420) του superblock βρίσκουμε τα blocks per group. Υπάρχουν 8192 blocks σε κάθε block group.

Ερώτηση 27

```
root@utopia:/mnt/fsdisk1# ls -li /mnt/fsdisk1/dir2/helloworld
9162 /mnt/fsdisk1/dir2/helloworld
```

Hexedit

Το πρώτο inode που πρέπει να βρούμε είναι αυτό που αντιστοιχεί στο αρχείο "/". Αυτό είναι γνωστό και είναι πάντα το inode 2. Θα διαβάσουμε το inode για να βρούμε το directory entry block του "/". Μεταβαίνουμε στο block 5 το οποίο είναι και η αρχή του inode table και διαβάζουμε το δεύτερο inode. Στο byte 40 βρίσκουμε το αντίστοιχο directory entry block, το αντίστοιχο offset υπολογίζεται ως εξής:

$$\text{offset} = 5 \cdot 1024 + 128 + 40 = 0x14A8$$

Βλέπουμε ότι το directory entry block είναι το 234. Πάμε στο offset $234 \cdot 1024 = 0x3A800$. Βλέπουμε ότι το inode του /dir2 είναι το 0x23C9 = 9161, το πρώτο inode του Block Group 5. Το inode table του Block Group 5 μπορεί να βρεθεί στο offset $40965 \cdot 1024 = 0x2801400$. Στο byte 40 βρίσκουμε το αντίστοιχο directory entry block, το 0xF7 = 247. Το αντίστοιχο offset είναι το 0x3DC00. Εντοπίζουμε την καταχώρηση του αρχείου helloworld και βλέπουμε ότι έχει inode 0x23CA = 9162.

Ερώτηση 28

Υπολογισμός Block Group:

Το block group για ένα inode υπολογίζεται ως εξής:

$$\text{Block Group} = \frac{\text{Inode Number} - 1}{\text{Inodes per Group}}$$

Από τα δεδομένα:

- **Inode Number:** 9162
- **Inodes per Group:** 1832 (από το αποτέλεσμα της **dumpe2fs**).

Υπολογίζουμε: $\text{Block Group} = \frac{9161}{1832} \simeq 5$. Το inode 9162 βρίσκεται στο **Block Group 5**.

Hexedit

Όπως είδαμε και παραπάνω, το inode αντιστοιχεί στο Block Group 5.

Ερώτηση 29

Για να προσδιορίσουμε σε ποιο **μπλοκ του δίσκου** βρίσκεται το **inode table** που περιέχει το inode **9162**, χρησιμοποιούμε τις πληροφορίες από την έξοδο της **dumpe2fs**.

Από την προηγούμενη ερώτηση, το inode **9162** βρίσκεται στο **Block Group 5**.

Θέση του Inode Table στο Block Group 5:

Στην έξοδο της **dumpe2fs**, βλέπουμε:

Group 5: Inode table at 40965-41193 (+4)

Αυτό σημαίνει ότι το **inode table** για το Block Group 5 ξεκινά στο **Block 40965** και εκτείνεται μέχρι το **Block 41193**.

Υπολογισμός της Θέσης του inode μέσα στο Inode Table:

1. Το κάθε Block Group περιέχει **1832 inodes** (από την έξοδο της **dumpe2fs**).
2. Τα inodes στο Block Group 5 ξεκινούν από: $\text{Start Inode} = (5 \cdot 1832) + 1 = 9161$
3. Το inode **9162** είναι το δεύτερο inode στο Block Group 5.

Θέση του Inode στο Inode Table:

- Κάθε μπλοκ αποθηκεύει **8 inodes** (γιατί το μέγεθος του inode είναι **128 bytes** και το block size είναι **1024 bytes**).
- Το δεύτερο inode βρίσκεται στο πρώτο μπλοκ του Inode Table για το Block Group 5.

Hexedit

Είναι το δεύτερο inode του Block Group 5, άρα βρίσκεται στο πρώτο block του inode table του Block Group 5, δηλαδή στο block 40965.

Ερώτηση 30

Η εντολή **stat dir2/helloworld** επέστρεψε τα εξής πεδία του inode:

```
root@utopia:/mnt/fsdisk1# stat dir2/helloworld
File: dir2/helloworld
Size: 42 Blocks: 2 IO Block: 1024 regular file
Device: 254,16 Inode: 9162 Links: 1
```

Access: (0644/-rw-r--r-) Uid: (0/ root) Gid: (0/ root)
Access: 2024-12-29 16:14:50.000000000 +0000
Modify: 2023-12-12 15:23:16.000000000 +0000
Change: 2023-12-12 15:23:16.000000000 +0000

Επεξήγηση:

Η εντολή **stat** εμφανίζει όλα τα βασικά πεδία που περιέχει το inode. Αυτά περιλαμβάνουν:

- **Size:** Το μέγεθος του αρχείου σε bytes.
- **Blocks:** Ο αριθμός των μπλοκ που χρησιμοποιούνται για την αποθήκευση του αρχείου.
- **Permissions (Access):** Τα δικαιώματα πρόσβασης στο αρχείο.
- **Uid και Gid:** Το αναγνωριστικό χρήστη και ομάδας που κατέχουν το αρχείο.
- **Timestamps:** Οι χρονικές σφραγίδες πρόσβασης, τροποποίησης, και αλλαγής του αρχείου.

Hexedit

```
02801480  A4 81 00 00 2A 00 00 00 B5 FA 75 67 E4 7A 78 65 E4 7A 78 65 00 00 00 00 00 00 01 00 02 00 00 00 .....*.ug.zxe.zxe.....
028014A0  00 00 00 00 01 00 00 00 01 04 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
028014C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
028014E0  00 00 00 00 BC F3 45 A3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....E.....
```

Ερώτηση 31

Το αρχείο αποθηκεύεται σε **2 μπλοκ**, όπως φαίνεται από την έξοδο:

Blocks: 2
Inode: 9162 Type: regular Mode: 0644 Flags: 0x0
Generation: 2739270588 Version: 0x00000001
User: 0 Group: 0 Size: 42
File ACL: 0
Links: 1 Blockcount: 2
Fragment: Address: 0 Number: 0 Size: 0
ctime: 0x65787ae4 – Tue Dec 12 15:23:16 2023
atime: 0x65787ae4 – Tue Dec 12 15:23:16 2023
mtime: 0x65787ae4 – Tue Dec 12 15:23:16 2023
BLOCKS:
(0):1025
TOTAL: 1

Hexedit

Διαβάζοντας το offset 40 του inode 9162, βλέπουμε ότι τα δεδομένα βρίσκονται στο block 0x401, δηλαδή στο 1025.

Ερώτηση 32

Το μέγεθος του αρχείου είναι **42 bytes**, όπως φαίνεται από την έξοδο:

Size: 42

Το μέγεθος αναφέρεται στον πραγματικό αριθμό bytes που περιέχει το αρχείο. Ωστόσο, το filesystem αποθηκεύει το αρχείο σε πολλαπλάσια του μεγέθους του block (1024 bytes), εξ ου και η χρήση 2 μπλοκ.

Hexedit

Διαβάζουμε τα bytes 4 έως 7 του inode. Το αρχείο έχει μέγεθος 0x2A = 42 bytes.

Ερώτηση 33

Η εντολή `cat /mnt/fsdisk1/dir2/helloworld` επέστρεψε το εξής περιεχόμενο:

```
root@utopia:/mnt/fsdisk1# cat /mnt/fsdisk1/dir2/helloworld
Welcome to the Mighty World of Filesystems
```

Hexedit

Μεταβαίνουμε στο block 1025, δηλαδή στο offset $1025 \cdot 1024 = 1049600 = 0x100400$.

```
001003f8  00 00 00 00 00 00 00 00 57 65 6c 63 6f 6d 65 20 74 6f 20 74 68 65 20 40 69 67 68 74 79 20 57 6f  ....Welcome to the Mighty Wo
00100418  72 6c 64 20 6f 66 20 46 69 6c 65 73 79 73 74 65 6d 73 00 00 00 00 00 00 00 00 00 00 00 00 00 00  rld of Filesystems.....
```

Η εικόνα fsdisk2.img

Ερώτηση 1

Παρομοίως, φορτώνουμε την εικόνα του δίσκου `fsdisk2.img` και την προσαρτούμε στο σύστημά μας:

```
root@utopia: # mount /dev/vdc /mnt/
```

Ερώτηση 2

Προσπαθούμε να δημιουργήσουμε ένα νέο αρχείο στο προσαρτημένο σύστημα αρχείων με την εντολή `touch`.

```
root@utopia:/mnt# touch file1
touch: cannot touch 'file1': No space left on device
```

Ερώτηση 3

Απέτυχε. Βλέπουμε το error message:

```
touch: cannot touch 'file1': No space left on device
```

Η αποτυχία της εντολής `touch` οφείλεται στο γεγονός ότι το σύστημα αρχείων δεν έχει διαθέσιμους πόρους (είτε χώρο είτε inodes), κάτι που θα εξετάσουμε στις επόμενες ερωτήσεις.

Ερώτηση 4

Για να κατανοήσουμε καλύτερα το σφάλμα, χρησιμοποιούμε την `strace`, η οποία μας δείχνει τις κλήσεις συστήματος που εκτελεί η `touch` και τον λόγο αποτυχίας της.

```
root@utopia:/mnt# strace touch /mnt/file1
```

```
openat(AT_FDCWD, "file1", O_WRONLY|O_CREAT|O_NOCTTY|O_NONBLOCK, 0666) = -1 ENOSPC (No space left on device)
```

Ερώτηση 5

Για να υπολογίσουμε τον συνολικό αριθμό αρχείων και καταλόγων στο σύστημα αρχείων, χρησιμοποιούμε την `find` και μετράμε τις καταχωρήσεις:

```
root@utopia:/mnt# find /mnt | wc -l
5127
```

Αρχεία:

```
root@utopia:/mnt# find /mnt -type f | wc -l
```

4868

Κατάλογοι:

```
root@utopia:/mnt# find /mnt -type d | wc -l
259
```

Ερώτηση 6

```
root@utopia:/mnt# du -sh /mnt
270K /mnt
```

Αυτό δείχνει ότι τα δεδομένα που είναι αποθηκευμένα στο σύστημα αρχείων καταλαμβάνουν 270 KB.

Ερώτηση 7

Για να δούμε το συνολικό μέγεθος του συστήματος αρχείων, χρησιμοποιούμε την df:

```
root@utopia:/mnt# df -h /mnt
Filesystem Size Used Avail Use% Mounted on
/dev/vdc 20M 270K 20M 2% /mnt
```

Το σύστημα αρχείων έχει συνολικό μέγεθος 20MB.

Ερώτηση 8

Για να βρούμε τα διαθέσιμα μπλοκ στο σύστημα αρχείων, χρησιμοποιούμε την dumpe2fs:

```
root@utopia:/mnt# dumpe2fs /dev/vdc | grep -i "free blocks"
dumpe2fs 1.47.0 (5-Feb-2023)
Free blocks: 19555
7787 free blocks, 0 free inodes, 84 directories
Free blocks: 406-8192
7891 free blocks, 0 free inodes, 83 directories
Free blocks: 8494-16384
3877 free blocks, 0 free inodes, 92 directories
Free blocks: 16603-20479
```

Το αποτέλεσμα μας δείχνει αρκετά διαθέσιμα μπλοκ, γεγονός που σημαίνει ότι υπάρχει ελεύθερος αποθηκευτικός χώρος.

Hexedit

Στο offset 12 του superblock μπορούμε να δούμε των αριθμό των μη ανατεθειμένων blocks. Διαβάζουμε 0x4C63 = 19555 ελεύθερα blocks. Συνεπώς, το σύστημα έχει ελεύθερο χώρο.

Ερώτηση 9

Παρόλο που υπάρχουν διαθέσιμα μπλοκ, η δημιουργία νέου αρχείου αποτυγχάνει. Εξετάζουμε αν υπάρχουν διαθέσιμα inodes:

```
root@utopia:/mnt# dumpe2fs /dev/vdc | grep -i "free inodes"
dumpe2fs 1.47.0 (5-Feb-2023)
Free inodes: 0
7787 free blocks, 0 free inodes, 84 directories
Free inodes:
7891 free blocks, 0 free inodes, 83 directories
Free inodes:
```


3877 free blocks, 0 free inodes, 92 directories

Free inodes:

Αυτό σημαίνει ότι το σύστημα αρχείων έχει εξαντλήσει όλα τα διαθέσιμα inodes. Παρόλο που υπάρχει αποθηκευτικός χώρος, δεν μπορεί να δημιουργηθεί νέο αρχείο επειδή δεν υπάρχουν ελεύθερα inodes για να το καταχωρήσουν.

Hexedit

Διαβάζοντας το Block Group Descriptor 0 βλέπουμε τις διευθύνσεις όλων των inode bit maps: 0x0004, 0x2004, 0x4004. Αν μεταβούμε στις διευθύνσεις αυτές θα δούμε ότι όλες οι θέσεις και στα τρία bit maps περιέχουν τον αριθμό F. Αυτό σημαίνει ότι όλα τα inodes χρησιμοποιούνται. Για αυτό δεν μπορούμε να δημιουργήσουμε καινούριο αρχείο, γιατί δεν υπάρχει inode που μπορούμε να δεσμεύσουμε.

Η εικόνα fsdisk3.img

Ερώτηση 1

Το εργαλείο που αναλαμβάνει τον έλεγχο και την επιδιόρθωση ενός συστήματος αρχείων **ext2** στο Linux είναι το **fsck.ext2** (File System Consistency Check for ext2). Είναι μέρος της οικογένειας εργαλείων **fsck** και είναι ειδικά σχεδιασμένο για τον έλεγχο της συνέπειας και τη διόρθωση προβλημάτων στο σύστημα αρχείων ext2.

Επιδιορθώνει προβλήματα, όπως:

- Κατεστραμμένα inodes.
- Μπλοκ που αναφέρονται από περισσότερα από ένα inodes.
- Μπλοκ που δεν αναφέρονται από κανένα inode.
- Ανακολουθίες στα bitmaps (block και inode).

```
root@utopia:~# fsck.ext2 /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img contains a file system with errors, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
First entry 'B00' (inode=1717) in directory inode 1717 (/dir-2) should be '.
'
Fix<y>? no
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Inode 3425 ref count is 1, should be 2. Fix<y>? no
Pass 5: Checking group summary information
Block bitmap differences: +34
Fix<y>? no
Free blocks count wrong for group #0 (7960, counted=7961).
Fix<y>? no
Free blocks count wrong (926431538, counted=19801).
Fix<y>? no
10 PM
fsdisk3.img: ***** WARNING: Filesystem still has errors *****
11 PM
fsdisk3.img: 23/5136 files (0.0% non-contiguous), 18446744072783140558/20480
blocks
```

Ερώτηση 2

1. **Απροσδόκητη απώλεια ισχύος** Αν το σύστημα τερματιστεί ξαφνικά κατά τη διάρκεια μιας εγγραφής, μπορεί να προκληθούν ασυνεπείς εγγραφές στο σύστημα αρχείων.
2. **Κατεστραμμένοι ή ελαττωματικοί τομείς του δίσκου (bad sectors)** Αν ένας τομέας του δίσκου που περιέχει σημαντικά μεταδεδομένα (π.χ. superbloc, inodes) καταστραφεί, το σύστημα αρχείων μπορεί να γίνει μη προσβάσιμο ή ασταθές.
3. **Αποτυχίες στον ελεγκτή του δίσκου (disk controller failure)** Ένας ελαττωματικός ελεγκτής μπορεί να οδηγήσει σε αλλοιώσεις στα δεδομένα κατά την ανάγνωση ή εγγραφή.
4. **Λάθη λογισμικού (software bugs)** Σφάλματα στο λειτουργικό σύστημα ή στον driver του συστήματος αρχείων μπορεί να προκαλέσουν καταστροφή δεδομένων ή μεταδεδομένων.
5. **Κακόβουλο λογισμικό (malware/viruses)** Ιοί ή ransomware μπορούν να τροποποιήσουν, διαγράψουν ή κρυπτογραφήσουν αρχεία, αλλοιώνοντας το σύστημα αρχείων.
6. **Εσφαλμένη διαμόρφωση (misconfiguration)** Εσφαλμένες ρυθμίσεις, όπως λάθος ορισμός δικαιωμάτων πρόσβασης ή επιλογές προσάρτησης, μπορεί να οδηγήσουν σε απρόβλεπτες συμπεριφορές.
7. **Αποτυχία RAID συστοιχίας** Αν ένα RAID σύστημα αποτύχει ή υπάρξει ασυνεπής εγγραφή μεταξύ των δίσκων, το σύστημα αρχείων μπορεί να αλλοιωθεί.
8. **Ακατάλληλο unmounting του συστήματος αρχείων** Αν ένα σύστημα αρχείων δεν αποπροσαρτηθεί σωστά (π.χ. εξαγωγή USB δίσκου χωρίς unmount), μπορεί να μείνουν ατελείς συναλλαγές δεδομένων.
9. **Κατεστραμμένα ή εσφαλμένα εργαλεία επισκευής (fsck, chkdsk)** Αν τα εργαλεία επιδιόρθωσης συστήματος αρχείων εκτελεστούν σε λάθος χρονική στιγμή ή με λανθασμένες παραμέτρους, μπορεί να προκαλέσουν επιπλέον αλλοιώσεις αντί να τις διορθώσουν.
10. **Υπερχείλιση χωρητικότητας** Αν εξαντληθεί ο διαθέσιμος χώρος αποθήκευσης, οι εγγραφές μπορεί να οδηγήσουν σε ανακρίβειες στα metadata.

Πιθανές αλλοιώσεις στο σύστημα αρχείων:

1. Μη έγκυρα inodes ή μπλοκ δεδομένων.
2. Αναφορά μπλοκ από πολλαπλά inodes.
3. Μπλοκ που δεν αναφέρονται από κανένα inode.
4. Ανακολουθίες στα bitmaps (block ή inode).
5. Λανθασμένοι αριθμοί hard links στα inodes.
6. Κατεστραμμένες χρονικές σφραγίδες αρχείων.
7. Κατεστραμμένα ή λανθασμένα δέντρα καταλόγων.
8. Λανθασμένες αναφορές σε pointers αρχείων.
9. Κατεστραμμένο superbloc ή block group descriptors.
10. Λάθη στη διαχείριση της ελεύθερης χωρητικότητας του δίσκου.

Ερώτηση 3

```
root@utopia:~# fsck.ext2 -n /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img contains a file system with errors, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
First entry 'B00' (inode=1717) in directory inode 1717 (/dir-2) should be '.'.
Fix? no

Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Inode 3425 ref count is 1, should be 2. Fix? no

Pass 5: Checking group summary information
Block bitmap differences: +34
Fix? no

Free blocks count wrong for group #0 (7960, counted=7961).
Fix? no

Free blocks count wrong (926431538, counted=19801).
Fix? no

fsdisk3.img: ***** WARNING: Filesystem still has errors *****

fsdisk3.img: 23/5136 files (0.0% non-contiguous), 18446744072783140558/20480
blocks
```

Αλλοιώσεις που εντοπίστηκαν και επιδιορθώθηκαν:

1. Λανθασμένη εγγραφή στον κατάλογο:

- Ο πρώτος καταχωρημένος φάκελος ενός καταλόγου (inode=1717 στον /dir-2) είχε ως όνομα το 'B00' αντί για το απαιτούμενο '.' (το οποίο υποδεικνύει τον εαυτό του καταλόγου).
- Διορθώθηκε αλλάζοντας το όνομα σε '.'.

2. Λανθασμένος reference count:

- Το inode 3425 είχε αναφορά μετρητή 1, ενώ έπρεπε να έχει 2.
- Διορθώθηκε με την ενημέρωση του reference count.

3. Λανθασμένες Πληροφορίες για το Block Bitmap:

- Εντοπίστηκε διαφορά στο bitmap των μπλοκ
- Αυτό σημαίνει ότι το σύστημα αρχείων έχει αποκλίσεις στην καταγραφή των χρησιμοποιούμενων και ελεύθερων μπλοκ.

4. Λανθασμένος αριθμός ελεύθερων μπλοκ:

- Εντοπίστηκε λανθασμένος αριθμός ελεύθερων μπλοκ για το Group #0
- Επίσης, συνολικά τα ελεύθερα μπλοκ είναι καταγεγραμμένα λανθασμένα
- Αυτά τα προβλήματα δείχνουν ότι το σύστημα αρχείων έχει λανθασμένες πληροφορίες για τα διαθέσιμα μπλοκ, πιθανώς λόγω ακατάλληλου τερματισμού ή αλλοίωσης στη δομή του superblock.

Επιδιόρθωση με τη μέθοδο hexedit

Η εκτέλεση της εντολής `fsck.ext2 -n /dev/vdd` για την εικόνα `fsdisk3.img` στην αρχική της μορφή δίνει το ακόλουθο αποτέλεσμα:

```
root@utopia:~# fsck.ext2 -n /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img contains a file system with errors, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
First entry 'BOO' (inode=1717) in directory inode 1717 (/dir-2) should be '.'
Fix? no

Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Inode 3425 ref count is 1, should be 2. Fix? no

Pass 5: Checking group summary information
Block bitmap differences: +34
Fix? no

Free blocks count wrong for group #0 (7960, counted=7961).
Fix? no

Free blocks count wrong (926431538, counted=19801).
Fix? no

fsdisk3.img: ***** WARNING: Filesystem still has errors *****

fsdisk3.img: 23/5136 files (0.0% non-contiguous), 18446744072783140558/20480 blocks
```

Όπως δείξαμε και προηγουμένως, εντοπίζουμε το directory entry block του inode 1717 και αντικαθιστούμε την καταχώρηση BOO με τρεις τελείες. Μετά από την αλλαγή αυτή η εντολή `fsck.ext2` επιστρέφει τα ακόλουθα:

```
root@utopia:~# fsck.ext2 -n /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img contains a file system with errors, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Inode 3425 ref count is 1, should be 2. Fix? no

Pass 5: Checking group summary information
Block bitmap differences: +34
Fix? no

Free blocks count wrong for group #0 (7960, counted=7961).
Fix? no

Free blocks count wrong (926431538, counted=19801).
Fix? no

fsdisk3.img: ***** WARNING: Filesystem still has errors *****

fsdisk3.img: 23/5136 files (0.0% non-contiguous), 18446744072783140558/20480 blocks
```

Για να φτιάξουμε το inode 3425 πρέπει να βρούμε πρώτα πού είναι αποθηκευμένο στη μνήμη. Από το superblock (offset 40) μαθαίνουμε ότι υπάρχουν 1712 inodes per group. Επομένως, είναι το πρώτο inode του Block Group 2. Σύμφωνα με την τρίτη καταχώρηση του Block Group 0, το inode table του Block Group 2 ξεκινάει από το block 0x4005. Μετά την αλλαγή του reference count (byte 26) από 1 σε 2. Μετά την αλλαγή του inode

3425 έχουμε:

```
root@utopia:~# fsck.ext2 -n /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img contains a file system with errors, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
Block bitmap differences: +34
Fix? no

Free blocks count wrong for group #0 (7960, counted=7961).
Fix? no

Free blocks count wrong (926431538, counted=19801).
Fix? no

fsdisk3.img: ***** WARNING: Filesystem still has errors *****
fsdisk3.img: 23/5136 files (0.0% non-contiguous), 18446744072783140558/20480 blocks
```

Πηγαίνουμε στο block bit map, block διεύθυνση 3, δηλαδή 0xC00. Το μήνυμα του σφάλματος ενημερώνει ότι το block 34 είναι σημειωμένο ως ελεύθερο, ενώ στην πραγματικότητα χρησιμοποιείται. Σημειώνουμε ότι το block bit map δεν απευθύνεται στο block 0, καθώς είναι το boot sector, συνεπώς η μέτρησή μας θα ξεκινήσει από το block 1. Εφόσον κάθε byte περιγράφει την κατάσταση που έχουν 8 blocks, το 34^ο block αντιπροσωπεύεται από το τρίτο LSB του πέμπτου byte. Πράγματι το πέμπτο byte είναι το FD, σε δυαδική μορφή 1111 1101. Μετατρέποντας το D σε F, έχουμε διορθώσει το σφάλμα.

```
root@utopia:~# fsck.ext2 -n /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img contains a file system with errors, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
Free blocks count wrong (926431538, counted=19800).
Fix? no

fsdisk3.img: 23/5136 files (0.0% non-contiguous), 18446744072783140558/20480 blocks
```

Το τελευταίο πρόβλημα αφορά τη μέτρηση των ελεύθερων block. Βλέπουμε ότι έχουν μετρηθεί 19800 ελεύθερα blocks. Πηγαίνουμε στο πρώτο superbloc στο offset 0x40C και τοποθετούμε τον αριθμό 0x4D58. Μετά από αυτή την αλλαγή δεν προκύπτουν άλλα λάθη.

```
root@utopia:~# fsck.ext2 -n /dev/vdd
e2fsck 1.47.0 (5-Feb-2023)
fsdisk3.img was not cleanly unmounted, check forced.
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
fsdisk3.img: 23/5136 files (0.0% non-contiguous), 680/20480 blocks
```

2^ο Μέρος

Γενική Επισκόπηση

Το ext2-lite ενσωματώνεται στον πυρήνα του Linux ως kernel module και αλληλεπιδρά με το VFS (Virtual File System), το οποίο επιτρέπει τη διαχείριση πολλαπλών συστημάτων αρχείων. Η προσάρτηση και αποπροσάρτηση πραγματοποιούνται μέσω των συναρτήσεων **ext2_mount** και **unregister_filesystem**.

Βασικές δομές:

- **ext2_sb_info**, **ext2_inode_info** (δομές στη μνήμη)
- **ext2_super_block**, **ext2_inode** (δομές στον δίσκο)

Αρχεία και ο Ρόλος τους

super.c

Το αρχείο **super.c** είναι υπεύθυνο για την αρχικοποίηση και αποδέσμευση του συστήματος αρχείων **ext2-lite**.

Αρχικοποίηση του Συστήματος Αρχείων

Το module ξεκινά με την **init_ext2_fs**, η οποία εκτελεί τις εξής ενέργειες:

1. Δημιουργία cache για inodes με **init_inodecache()**, που διαχειρίζεται τα inodes στη μνήμη.

```
int err = init_inodecache()...

kmem_cache_create_usercopy("ext2_inode_cache", sizeof(struct ext2_inode_info), 0, | SLAB_ACCOUNT, offsetof(struct ext2_inode_info, i_data), sizeof_field(struct ext2_inode_info, i_data), init_once);
```

2. Η συνάρτηση **init_once** εξασφαλίζει τη σωστή αρχικοποίηση του **vfs_inode**:

```
static void init_once(void *foo){
    struct ext2_inode_info *ei = (struct ext2_inode_info *)foo;    inode_init_once(&ei->vfs_inode);
}
```

3. Καταχώρηση του Συστήματος Αρχείων: Η **register_filesystem** προσθέτει το **ext2-lite** στο VFS, χρησιμοποιώντας την παρακάτω δομή

```
static struct file_system_type ext2_fs_type = {
    .owner = THIS_MODULE,
    .name = "ext2-lite",
    .mount = ext2_mount,
    .kill_sb = kill_block_super,
    .fs_flags = FS_REQUIRES_DEV,
};
```

4. Η κύρια συνάρτηση αρχικοποίησης και ο κώδικας που γράψαμε:

```
static int __init init_ext2_fs(void){
    /* Register ext2-lite filesystem in the kernel */
    /* If an error occurs remember to call destroy_inodecache() */
    /*----- OUR CODE -----*/
    err = register_filesystem(&ext2_fs_type);
    // Use the VFS support of registering filesystems
    if (err){
        destroy_inodecache();
        return err;
    } return err;
    /*-----*/
}
```

Αν η εγγραφή αποτύχει, η `destroy_inodecache()` καθαρίζει τη μνήμη:

```
static void destroy_inodecache(void){
    rcu_barrier();
    kmem_cache_destroy(ext2_inode_cache);
}
```

Επεξήγηση: Καταχωρούμε το `&ext2_fs_type` τύπο στο σύστημά μας και αποθηκεύουμε τον κωδικό επιστροφής. Εάν έχουμε επιτυχία `err=0` τότε επιστρέφουμε 0 αλλιώς επειδή απέτυχε η καταχώρηση καταστρέφουμε το `inodecache` και επιστρέφουμε τον αριθμό σφάλματος.

Αποδέσμευση του Συστήματος Αρχείων

Η αποδέσμευση του `ext2-lite` γίνεται μέσω της `exit_ext2_fs`: (Καταργεί το σύστημα αρχείων (`unregister_filesystem`) και αποδεσμεύει `cache inodes` (`destroy_inodecache`)).

```
static void __exit exit_ext2_fs(void){
    /* Unregister ext2-lite filesystem from the kernel */
    /*----- OUR CODE -----*/
    unregister_filesystem(&ext2_fs_type); // VFS supported function
    /*-----*/
    destroy_inodecache();
}
```

Διαχείριση Superblock:

Η `ext2_fill_super` αρχικοποιεί το superblock του συστήματος αρχείων:

- Δέσμευση μνήμης για `ext2_sb_info`.
- Ορισμός `blocksize`.
- Εντοπισμός και ανάγνωση του superblock από τον δίσκο μέσω `sb_bread`.
- Φόρτωση `group descriptors`.
- Καταχώρηση της `ext2_sops` στο πεδίο `s_op` του VFS superblock.
- Δημιουργία `root inode` και αποθήκευσή του στο `s_root`.

Η διαδικασία αυτή υλοποιείται μέσω `mount_bdev` που καλείται από την `ext2_mount`:

```
ext2_mount...

return mount_bdev(fs_type, flags, dev_name, data, ext2_fill_super);
```

Η δομή `ext2_sops` περιέχει τις βασικές λειτουργίες του superblock:

```
static const struct super_operations ext2_sops = {
    .alloc_inode = ext2_alloc_inode,
    .free_inode = ext2_free_inode_sb,
    .put_super = ext2_put_super,
    .sync_fs = ext2_sync_fs,
    .statfs = ext2_statfs,
    .remount_fs = ext2_remount,
    .show_options = ext2_show_options,
    .write_inode = ext2_write_inode,
    .evict_inode = ext2_evict_inode,
};
```

Η `ext2_put_super` αποθηκεύει τις αλλαγές του superblock στον δίσκο κατά την αποπροσάρτηση του συστήματος αρχείων.

inode.c

Το αρχείο inode.c διαχειρίζεται τα inodes στο ext2-lite. Μετά την προσάρτηση του συστήματος αρχείων, ο πυρήνας αποκτά πρόσβαση στο root dentry, το οποίο παρέχει το root inode.

Ανάκτηση και Διαχείριση Inodes

Η συνάρτηση 'ext2_iget' είναι υπεύθυνη για την ανάκτηση ενός inode από τον δίσκο:

1. **Δέσμευση μνήμης για struct inode μέσω iget_locked, που καλεί τη alloc_inode του ext2_sops:**

```
inode = iget_locked(sb, ino);
```

Η EXT2_I χρησιμοποιεί τη container_of του πυρήνα για την επιστροφή του ext2_inode_info που περιέχει το inode του VFS:

```
struct ext2_inode_info *ei = EXT2_I(inode);
```

2. **Διάβασμα του ext2 inode από τον δίσκο με χρήση της ext2_get_inode:**

```
raw_inode = ext2_get_inode(inode->i_sb, ino, &bh);
```

ext2_get_inode: Εντοπίζει και επιστρέφει δεδομένα inode από τον δίσκο. Η ext2_get_inode εντοπίζει το block group και διαβάζει τον group descriptor. Με βάση αυτόν, υπολογίζει το block όπου αποθηκεύεται το inode και το διαβάζει από την page cache:

```
/*----- OUR CODE -----*/
/* Figure out in which block is the inode we are looking for and get its group block
   descriptor. */
block_group=(ino-1)/inodes_pg; // adjusted by 1 because inode numbers are 1-based
offset = (ino - 1) % inodes_pg;
/* Figure out the offset within the block group inode table */
/* Return the pointer to the appropriate ext2_inode */
/* Determine the block group and offset within the group */
block_group = (ino - 1) / inodes_pg;
offset = (ino - 1) % inodes_pg;
/* Get the group descriptor for the block group */
gdp = ext2_get_group_desc(sb, block_group, NULL);
if (!gdp)
    goto eioval;
/* Determine the block containing the inode table */
block = le32_to_cpu(gdp->bg_inode_table) + (offset * inode_sz) / blocksize;
/* Read the block from the inode table */
bh = sb_bread(sb, block);
if (!bh)
    goto eio;
*p = bh;
/* Calculate the exact location of the inode within the block */
offset = (offset * inode_sz) % blocksize;
return (struct ext2_inode *) (bh->b_data + offset);
/*-----*/
```

3. **Αρχικοποίηση του struct inode του VFS:** Τα πεδία i_op, i_fop, και i_mapping->a_ops ρυθμίζονται ανάλογα με τον τύπο του αρχείου:

For a Regular File:

```
if (S_ISREG(inode->i_mode)) /* Is a Regular File? */ {
    /*----- OUR CODE -----*/
    /* Functions defined in the ext2.h and used at namei.c as well */
    inode->i_op = &ext2_file_inode_operations;
    inode->i_fop = &ext2_file_operations;
    inode->i_mapping->a_ops = &ext2_aops; // @a_ops: Methods.
    /*-----*/
}
```


For a Directory:

```
else if (S_ISDIR(inode->i_mode)) /* Is a Directory */{
    /* ----- OUR CODE -----
    ----- */
    inode->i_op = &ext2_dir_inode_operations;
    inode->i_fop = &ext2_dir_operations;
    inode->i_mapping->a_ops = &ext2_aops; // ext2_aops: Defined in
    this file
    /* -----
    */
}
```

Δημιουργία και Απελευθέρωση Inodes

Η διαχείριση των inodes περιλαμβάνει τις εξής συναρτήσεις:

- ext2_alloc_inode: Δέσμευση μνήμης για νέο inode.
- ext2_free_inode_sb: Απελευθέρωση inode.

Αυτές οι λειτουργίες εξασφαλίζουν την αποδοτική διαχείριση των 'inodes' στη μνήμη.

Ροή κώδικα

1. Αρχικοποίηση του root inode μέσω ext2_mount:

- Όταν προσαρτάται ένα σύστημα αρχείων ext2-lite, ο πυρήνας αποκτά πρόσβαση στο root dentry, το οποίο παρέχει το root inode.
- Το root inode αρχικοποιείται μέσα από τη συνάρτηση ext2_fill_super, που καλεί τη ext2_iget.

2. Διαδικασία της ext2_iget:

- **Δέσμευση μνήμης για struct inode:**
 - Χρησιμοποιείται η συνάρτηση iget_locked, η οποία καλεί τη μέθοδο .alloc_inode του ext2_sops για τη δέσμευση μνήμης.
 - Το struct inode περιλαμβάνεται στο ext2_inode_info.
 - Μετατροπή του δείκτη γίνεται μέσω της EXT2_I, η οποία χρησιμοποιεί τη συνάρτηση container_of για επιστροφή του κατάλληλου δείκτη.
- **Αρχικοποίηση του struct inode του VFS**
 - Ορίζονται τα πεδία i_op, i_fop, και i_mapping->a_ops ανάλογα με τον τύπο του αρχείου που αφορά το inode.
- **Αρχικοποίηση του struct ext2_inode_info:**
 - Εισάγονται δεδομένα που είναι ειδικά για το ext2 filesystem.
- **Απελευθέρωση πόρων:**
 - Μειώνεται ο μετρητής αναφοράς του buffer_head που χρησιμοποιήθηκε για το διάβασμα.
 - Επιστρέφεται το inode για περαιτέρω χρήση.

dir.c

Το αρχείο **dir.c** διαχειρίζεται τις λειτουργίες των καταλόγων στο ext2-lite.

Εύρεση Καταχωρήσεων Καταλόγου (Directory Entries)

Η ext2_dirent χρησιμοποιείται για την αποθήκευση και την αναζήτηση αρχείων στους καταλόγους. Η συνάρτηση ext2_find_entry αναζητά μια συγκεκριμένη καταχώρηση σε όλες τις σελίδες του καταλόγου και επιστρέφει έναν δείκτη στην αντίστοιχη καταχώρηση αν βρεθεί, διαφορετικά επιστρέφει σφάλμα.

```

ext2_dirent *ext2_find_entry(struct inode *dir, const struct qstr *child, struct
    folio **foliop){
    const char *name = child->name;
    int namelen = child->len;
    unsigned reclen = EXT2_DIR_REC_LEN(namelen);
    unsigned long npages = dir_pages(dir);
    unsigned long i;
    ext2_dirent *de;
    if (npages == 0)
        return ERR_PTR(-ENOENT);
    for (i = 0; i < npages; i++){
        /*----- OUR CODE -----*/
        char *kaddr = ext2_get_folio(dir, i, 0, foliop);
        if (IS_ERR(kaddr))
            continue; /* Skip to the next page if there is an error */
        /* Iterate through the directory entries in the current page */
        char *limit = kaddr + ext2_last_byte(dir, i) - reclen;
        for (de = (ext2_dirent *)kaddr; (char *)de <= limit; de = ext2_next_entry(de)){
            if (de->rec_len == 0){
                ext2_error(dir->i_sb, __func__, "zero-length directory entry");
                folio_release_kmap(*foliop, kaddr);
                return ERR_PTR(-ENOENT);
            }
            /* Match the name with the current directory entry */
            if (ext2_match(namelen, name, de)){
                folio_release_kmap(*foliop, kaddr); /* Release before returning */
                return de; /* Entry found */
            }
        }
        folio_release_kmap(*foliop, kaddr); /* Release mapping for this page */
    }
    /*-----*/
    return ERR_PTR(-ENOENT);
}

```

Επεξήγηση Κώδικα:

- Η `dir_pages(dir)` επιστρέφει τον αριθμό των σελίδων στον κατάλογο.
- Γίνεται αναζήτηση σε κάθε σελίδα (for loop) με χρήση της `ext2_get_folio`.
- Αν η σελίδα περιέχει μη έγκυρη καταχώρηση (`rec_len == 0`), επιστρέφεται σφάλμα.
- Αν η `ext2_match` βρει όνομα που ταιριάζει, επιστρέφεται η καταχώρηση.
- Τέλος, η `folio_release_kmap` απελευθερώνει τη χαρτογράφηση της σελίδας.

namei.c

Βασικές Ενέργειες:

- `ext2_create`: Δημιουργεί νέο αρχείο σε κατάλογο.
- `ext2_lookup`: Εντοπίζει ένα αρχείο στον κατάλογο.
- `ext2_unlink`: Διαγράφει όνομα αρχείου από κατάλογο.
- `ext2_mkdir`: Δημιουργεί νέο κατάλογο.

ballo.c

Το αρχείο `ballo.c` διαχειρίζεται τη δέσμευση και αποδέσμευση των blocks στο σύστημα αρχείων `ext2-lite`.

Δέσμευση και Απελευθέρωση Blocks

- `ext2_new_blocks`: Εντοπίζει και δεσμεύει διαθέσιμο block στον δίσκο.
- `ext2_free_blocks`: Απελευθερώνει blocks κατά τη διαγραφή δεδομένων.
- `ext2_count_free_blocks`: Μετρά διαθέσιμα blocks μέσω των group descriptors.

Δέσμευση Blocks σε Block Group

Η συνάρτηση `ext2_allocate_in_bg` εντοπίζει και δεσμεύει διαθέσιμα blocks μέσα σε ένα block group:

```
static int ext2_allocate_in_bg(struct super_block *sb, int group, struct buffer_head
    *bitmap_bh, unsigned long *count){
    ext2_fsblk_t group_first_block = ext2_group_first_block_no(sb, group);
    ext2_fsblk_t group_last_block = ext2_group_last_block_no(sb, group);
    ext2_grpblk_t nblocks = group_last_block - group_first_block + 1;
    ext2_grpblk_t first_free_bit;
    unsigned long num;
    unsigned long num_to_allocate = *count;
    unsigned long num_allocated = 0; // Tracks the number of allocated blocks
    unsigned long i; // Counter for "for loops"
    /*----- OUR CODE -----*/
    /* Find the first free bit in the bitmap */
    first_free_bit = find_next_zero_bit_le(bitmap_bh->b_data, nblocks, 0);
    if (first_free_bit >= nblocks)
        return -1; /* No free blocks found in this group */
    /* Check if we have enough contiguous blocks */
    for(i = first_free_bit; i < first_free_bit + num_to_allocate && i < nblocks; i++){
        if (test_bit_le(i, bitmap_bh->b_data))
            break; /* Found an allocated block, cannot continue */
        num_allocated++;
    }
    /* If we couldn't allocate the requested number of blocks */
    if (num_allocated < num_to_allocate)
        *count = num_allocated; /*Return the actual number of allocated blocks */
    else
        *count = num_to_allocate; /*Successfully allocated all requested blocks */
        /*Mark the blocks as allocated in the bitmap */
        for (i = first_free_bit; i < first_free_bit + *count; i++){
            ext2_set_bit_atomic(sb_bgl_lock(EXT2_SB(sb), group), i, bitmap_bh->b_data);
        }
        /* Mark the bitmap buffer as dirty to reflect the changes */
        mark_buffer_dirty(bitmap_bh);
    /* Return the group-relative block number of the first allocated block */
    return first_free_bit;
    /*-----*/
}
```

Επεξήγηση κώδικα:

- `find_next_zero_bit_le`: Βρίσκει το πρώτο διαθέσιμο block στο bitmap
- `test_bit_le`: Ελέγχει αν το bit ενός block είναι ήδη δεσμευμένο.
- `ext2_set_bit_atomic`: Δεσμεύει το block με ασφάλεια, αποτρέποντας ανταγωνιστικές εγγραφές.
- `sb_bgl_lock`: Εξασφαλίζει ατομικές ενημερώσεις στο χάρτη bitmap για την ασφάλεια των νημάτων.
- `mark_buffer_dirty`: Επισημαίνει το bitmap ως αλλαγμένο, ώστε να γραφεί στον δίσκο.

ialloc.c

Το αρχείο `ialloc.c` είναι υπεύθυνο για τη διαχείριση των inodes στο σύστημα αρχείων ext2-lite.

Δημιουργία/Διαγραφή:

- `ext2_new_inode`: Δεσμεύει νέο inode για αρχείο.
- `ext2_free_inode`: Αποδεσμεύει ένα inode και το επιστρέφει στο σύστημα για επαναχρησιμοποίηση.

Στατιστικά:

- `ext2_count_free_inodes`: Μετρά διαθέσιμα inodes.
- `ext2_count_dirs`: Υπολογίζει τον αριθμό των καταλόγων.

file.c

Το αρχείο `file.c` διαχειρίζεται τις βασικές λειτουργίες των αρχείων, όπως ανάγνωση και εγγραφή.

Λειτουργίες:

- Χρήση γενικών συναρτήσεων πυρήνα (`generic_file_read_iter`, `generic_file_write_iter`).
- Ειδική υποστήριξη για καταλόγους με τη `ext2_dir_operations`.

ext2.h

Το `ext2.h` είναι η κοινή βιβλιοθήκη που περιλαμβάνει όλες τις βασικές δομές δεδομένων και τις δηλώσεις συναρτήσεων που χρησιμοποιούνται σε όλο το σύστημα αρχείων `ext2-lite`.

Ροή Εντολών

Φόρτωση Module (insmod):

Η εκκίνηση του `ext2-lite` ξεκινά με την κλήση της `init_ext2_fs`, η οποία:

- Καταχωρεί το `ext2-lite` ως σύστημα αρχείων στο VFS
- Δημιουργεί cache για τα inodes μέσω `init_inodecache`.

Προσάρτηση (mount):

Η συνάρτηση `ext2_mount` καλεί τη `mount_bdev`, η οποία:

- Εντοπίζει και φορτώνει το superblock μέσω `ext2_fill_super`.
- Διαβάζει τους group descriptors από τον δίσκο.
- Δημιουργεί το root inode και το αποθηκεύει στο `s_root`.
- Χρησιμοποιεί τις λειτουργίες του `ext2_sops` για τη διαχείριση του superblock.

Προσπέλαση Αρχείων/Φακέλων:

VFS αλληλεπιδρά με τα αρχεία και τους καταλόγους μέσω των `inode_operations`:

- `ext2_dir_inode_operations` για καταλόγους.
- `ext2_file_inode_operations` για αρχεία.

Διαχείριση Blocks/Inodes:

Συναρτήσεις όπως `ext2_new_inode` και `ext2_new_blocks` χρησιμοποιούνται για τη δέσμευση χώρου στον δίσκο.

Αποπροσάρτηση (umount):

- Καλείται η `ext2_put_super` για αποθήκευση αλλαγών.
- Το module αφαιρείται με την `exit_ext2_fs`.

Παράδειγμα προσάρτησης στο utopia:

```
root@utopia:/home/user# mount -t ext2-lite -o loop ./ext2-lite.img /mnt
root@utopia:/home/user# cat /proc/mounts | grep ext2-lite
/dev/loop0 /mnt ext2-lite rw,relatime,errors=continue 0 0
root@utopia:/home/user# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0  128M  0 loop /mnt
sr0          11:0     1 1024M  0 rom
vda          253:0    0   10G  0 disk
└─vda1       253:1    0   10G  0 part /
vdb          253:16   0    50M  0 disk
vdc          253:32   0    20M  0 disk
vdd          253:48   0    20M  0 disk
root@utopia:/home/user#
```